

Aim:- Install Jscript/Cryptool tool (or any other equivalent) and demonstrate Asymmetric, Symmetric crypto algorithm, Hash and Digital/PKI signatures.

Tool Used: OpenSSL (CLI)

Equivalent Tools: CryptoTool (GUI), GnuPG (CLI), VeraCrypt (GUI), Python cryptography library.

1. Symmetric Encryption & Decryption (AES-256-CBC)

Concept: Same key is used to encrypt and decrypt.

Algorithm: AES-256-CBC (Advanced Encryption Standard, 256-bit key, Cipher Block Chaining mode).

Step 1: Create a file to encrypt

```
echo "This is my secret message." > secret.txt
```

Step 2: Encrypt the file

```
openssl enc -aes-256-cbc -salt -in secret.txt -out secret.enc
```

- **-aes-256-cbc** → Algorithm
- **-salt** → Adds randomness to encryption
- **-in** → Input file
- **-out** → Output encrypted file

You'll be asked for a password — **remember it.**

Step 3: Check if file is encrypted

```
cat secret.enc
```

- If you see **gibberish characters**, encryption worked.
- If you see readable text, something went wrong.

Step 4: Decrypt the file

```
openssl enc -aes-256-cbc -d -in secret.enc -out decrypted.txt
```

- **-d** → Decrypt mode

Step 5: Verify decryption

```
cat decrypted.txt
```

- Should display:

This is my secret message.

2. Asymmetric Encryption & Decryption (RSA)

Concept: Public key encrypts, private key decrypts.

Algorithm: RSA 2048-bit.

Step 1: Generate RSA key pair

```
openssl genrsa -out private.pem 2048  
openssl rsa -in private.pem -pubout -out public.pem
```

Step 2: Encrypt using the public key

```
openssl rsautl -encrypt -inkey public.pem -pubin -in secret.txt -out secret_rsa.enc
```

Step 3: Check if encrypted

```
cat secret_rsa.enc
```

- Should be binary/gibberish.

Step 4: Decrypt using the private key

```
openssl rsautl -decrypt -inkey private.pem -in secret_rsa.enc -out rsa_decrypted.txt
```

Step 5: Verify decryption

```
cat rsa_decrypted.txt
```

- Should match original message.

3. Hashing (SHA-256)

Concept: One-way function — can't be reversed.

Step 1: Create hash

```
openssl dgst -sha256 secret.txt
```

Example output:

SHA256(secret.txt)= 3a7bd3e2360a3d...

Step 2: Test change detection

```
echo "new line" >> secret.txt
```

```
openssl dgst -sha256 secret.txt
```

- Hash will be **completely different**.

4. Digital Signatures (PKI)

Concept: Proves authenticity and integrity.

Step 1: Sign a file

```
openssl dgst -sha256 -sign private.pem -out signature.bin secret.txt
```

Step 2: Verify the signature

```
openssl dgst -sha256 -verify public.pem -signature signature.bin secret.txt
```

OutPut:

Verified OK

If file is changed:

Verification Failure